LLM EVALUATION & BENCHMARKING PLATFORM

Software Architecture Document

(SAD)

Document Version 1.0
September 2026

Prepared by
Monish D — 24BDS1033
Soaham S — 24BDS1021

Project: LLM Evaluation and Benchmarking Platform
Course: Software Engineering — Digital Assignment

Table of Contents

Table of Contents 2
1. Introduction 3
1.1 Purpose 3
1.2 Architectural Goals 3
2. Architectural Style 3
3. Component View 3
3.1 Frontend (React, generated via Stitch / UI UX Pro Max AgentKit) 3
3.2 Backend API (FastAPI) 3
3.3 Benchmark Manager 3
3.4 LLM Integration Layer (Adapters) 3
3.5 Evaluation Engine 4
3.6 Industry Comparison Engine (Differentiating Module) 4
3.7 Reporting & Visualization Layer 4
3.8 Database 4
4. Data Flow 4
5. Technology Stack 4
6. Deployment View 5
7. Design Decisions & Rationale 5
8. Future Architectural Extensions 5

1. Introduction

1.1 Purpose

This Software Architecture Document (SAD) describes the structural design of the LLM Evaluation and Benchmarking Platform: its components, their responsibilities, the data flow between them, technology choices, and the deployment topology, all under a zero-cost infrastructure constraint. It elaborates the architecture summarized in the source project report into concrete module boundaries, particularly for the platform's differentiating capability — comparing evaluated models against industry-standard baselines and profiling per-category strengths/weaknesses.

1.2 Architectural Goals

- Modularity: provider adapters, evaluation metrics, and the comparison engine must be independently replaceable.
- Extensibility: new providers, datasets, capability categories, or metrics must not require core rewrites.
- Fault isolation: one model/provider failure must not corrupt or halt a run.
- Reproducibility: every run's configuration and every baseline score must be versioned and timestamped.
- Zero-cost operability: every component must have a free-tier deployment path.

2. Architectural Style

The system uses a layered, modular-monolith architecture for the backend (simpler to deploy for free on Railway than a microservices topology) with clear internal module boundaries that mirror what would be separate services if the project later scales: LLM Integration Layer, Evaluation Engine, Industry Comparison Engine, Benchmark Manager, and Reporting Layer, all behind a single FastAPI application, backed by PostgreSQL, with a decoupled React frontend.

3. Component View

3.1 Frontend (React, generated via Stitch / UI UX Pro Max AgentKit)

Provides model/dataset selection, run configuration, live progress, the comparative dashboard, capability radar visualizations, response drill-down, and report export. Communicates with the backend exclusively over REST (+ a polling or WebSocket channel for run progress). Holds no API keys and no evaluation logic.

3.2 Backend API (FastAPI)

Exposes REST endpoints for provider/model management, dataset management, run orchestration, results retrieval, and report generation. Performs request validation, authentication of API-key storage, and delegates execution to the Benchmark Manager.

3.3 Benchmark Manager

Coordinates a run: resolves the dataset, the selected models, and the shared parameters into a task matrix (tasks × models), enqueues each cell as an asynchronous unit of work, tracks per-cell status (pending/running/succeeded/failed), and marks the run complete only once all cells resolve (success or terminal failure).

3.4 LLM Integration Layer (Adapters)

One adapter per provider (OpenRouter, Gemini, NVIDIA NIM, Cloudflare Workers AI, self-hosted/OpenAI-compatible endpoints) implementing a common interface: call(prompt, params) → {text, input_tokens, output_tokens, latency_ms, raw}. Adapters own provider-specific auth, rate-limit backoff, and error normalization so the rest of the system never sees provider-specific exceptions.

3.5 Evaluation Engine

Composed of independent scorer modules invoked per RunResult: (a) Quality Scorer — exact-match/F1/semantic-similarity for closed-form tasks, LLM-as-a-Judge for open-ended tasks; (b) Hallucination & Reliability Scorer — claim extraction, claim verification against reference/source, groundedness, consistency; (c) Efficiency Scorer — latency/token aggregation; (d) Cost Scorer — pricing-table lookup and cost computation. Each scorer is independently unit-testable and independently swappable.

3.6 Industry Comparison Engine (Differentiating Module)

Consumes the Evaluation Engine's per-task scores plus the maintained Baseline Set (versioned, date-stamped reference scores for well-known industry-standard models per capability category) and produces: (i) a normalized score per model per capability category (e.g., z-score/percentile against the baseline distribution), (ii) a Tier label (Frontier/Strong/Mid/Weak) via configurable thresholds, (iii) a capability profile (vector across all categories, used for the radar chart), and (iv) an auto-generated plain-language positioning summary. This module is deliberately isolated from the Evaluation Engine so the Baseline Set can be refreshed or extended without touching scoring logic.

3.7 Reporting & Visualization Layer

Aggregates run results, capability profiles, and comparison output into dashboard payloads for the frontend and into exportable PDF/CSV reports.

3.8 Database

PostgreSQL (Supabase/Neon free tier) storing Provider, Model, Dataset, Task, BenchmarkRun, RunResult, EvaluationScore, CapabilityCategory, BaselineModel, BaselineScore, and Report entities, as introduced in the SRS Data Requirements section.

4. Data Flow

User configures run (models + dataset + params) → Backend API creates a BenchmarkRun → Benchmark Manager builds the task matrix and dispatches calls through the LLM Integration Layer → raw responses persist as RunResult rows → Evaluation Engine scorers consume RunResult rows and write EvaluationScore rows → Industry Comparison Engine consumes EvaluationScore rows plus BaselineScore rows and writes normalized CapabilityProfile/Tier data → Reporting Layer aggregates everything for the dashboard and export → Frontend renders tables, charts, capability radars, and positioning statements.

5. Technology Stack

Layer	Technology
Frontend	React + TypeScript, Recharts/Chart.js for charts, generated/scaffolded via Stitch (UI UX Pro Max AgentKit) + Codex
Backend API	Python, FastAPI, Pydantic for schema validation, async task execution (asyncio / background tasks or a lightweight queue)

Layer	Technology
Evaluation & Comparison logic	Python (scikit-learn/numpy for statistical normalization, embedding libraries for semantic similarity)
Database	PostgreSQL (Supabase or Neon free tier) via SQLAlchemy ORM; SQLite for local development
LLM providers	OpenRouter (free models), Google Gemini free tier, NVIDIA NIM free credits, Cloudflare Workers AI free tier, Groq free tier
Hosting	Railway (primary deployment target) for backend + frontend

6. Deployment View

Two deployable units on Railway: (1) the FastAPI backend service, connected to an external managed free Postgres instance (Supabase/Neon), with environment-variable-based secrets for provider API keys; (2) the React frontend, built as a static bundle and served either from Railway's static hosting or a lightweight Node/Nginx service, configured with the backend's public URL. Both services are defined via a railway.json / Procfile-equivalent so deployment is reproducible from the GitHub repository.

7. Design Decisions & Rationale

Decision	Rationale
Modular monolith over microservices	A two-person academic team with zero infrastructure budget cannot operate multiple independently-scaled services; internal module boundaries preserve future separability.
Baseline scores pre-computed/cached rather than always live	Avoids repeatedly paying (in quota or cost) to re-run expensive baseline models on every user run; keeps comparisons fast.
Adapters behind a common interface	Lets the free-tier provider list change over time (a real risk given quota volatility) without touching evaluation or comparison logic.
Comparison Engine isolated from Evaluation Engine	Lets the "industry standard" definition evolve (new baseline models, adjusted tier thresholds) independently of how raw scores are computed.
Every score traceable to method + baseline version + date	Directly satisfies the reproducibility and evaluation-accuracy non-functional requirements from the SRS.

8. Future Architectural Extensions

- Swap background-task execution for a proper queue (e.g., Redis-backed) if run volume grows beyond what Railway's free tier can hold in-process.
- Support locally-hosted open-source model evaluation via a pluggable local-inference adapter.
- Add a scheduled job to auto-refresh the Baseline Set from public leaderboards where licensing permits.